

LAPORAN PRAKTIKUM

INHERITANCE, GENERALIZATION & OVERRIDING

Modul Praktikum 4 - Pemrograman Berorientasi Objek

Diajukan untuk memenuhi salah satu tugas praktikum Pemrograman Berorientasi objek



Disusun Oleh:

Salman alfarisi firdaus (251511030)

2A D3

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2026

A. Bagian A - Circle dan Cylinder

Pada bagian ini, diimplementasikan relasi inheritance antara class Circle sebagai superclass (induk) dan class Cylinder sebagai subclass (turunan). Cylinder memperluas fungsionalitas Circle dengan menambahkan atribut tinggi (height) serta mengkhususkan perhitungan luas permukaan dan volume tabung.

1. Potongan Kode Inti (Circle, Cylinder, dan TestCylinder)

Kode: Circle.java

```
public class Circle {
    private double radius;
    private String color;

    public Circle() {
        radius = 1.0;
        color = "red";
    }

    public Circle(double radius) {
        this.radius = radius;
        color = "red";
    }

    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }
    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public double getArea() { return radius * radius * Math.PI; }

    @Override
    public String toString() {
        return "Circle[radius=" + radius + ", color=" + color + "];"
    }
}
```

Kode: Cylinder.java

```
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
```

```

        super(radius);
        this.height = height;
    }

    public double getHeight() { return height; }
    public void setHeight(double height) { this.height = height; }

    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height + 2 * Math.PI * getRadius() *
getRadius();
    }

    public double getVolume() {
        return super.getArea() * height;
    }

    @Override
    public String toString() {
        return "Cylinder: subclass of " + super.toString() + ", height=" + height;
    }
}

```

Kode: TestCylinder.java

```

public class TestCylinder {
    public static void main(String[] args) {
        Cylinder c1 = new Cylinder();
        Cylinder c2 = new Cylinder(10.0);
        Cylinder c3 = new Cylinder(2.0, 10.0);

        printCylinder("c1", c1);
        printCylinder("c2", c2);
        printCylinder("c3", c3);
        System.out.println(c3);
    }

    private static void printCylinder(String name, Cylinder c) {
        System.out.println(name);
        System.out.println("radius    = " + c.getRadius());
        System.out.println("height   = " + c.getHeight());
        System.out.println("base area = " + c.getArea());
        System.out.println("volume   = " + c.getVolume());
        System.out.println();
    }
}

```

2. Output Eksekusi TestCylinder

Program TestCylinder menguji pembentukan tiga objek Cylinder (c1 dengan parameter default, c2 dengan tinggi spesifik, dan c3 dengan radius dan tinggi spesifik).

```

c1
radius    = 1.0
height    = 1.0
base area = 12.566370614359172
volume    = 3.141592653589793

c2
radius    = 1.0
height    = 10.0
base area = 69.11503837897544
volume    = 31.41592653589793

c3
radius    = 2.0
height    = 10.0
base area = 150.79644737231007
volume    = 125.66370614359172

```

Gambar A.1: Output Eksekusi Program TestCylinder

3. Bukti Constructor Chain

Ketika objek subclass (Cylinder) diinstansiasi, constructor superclass (Circle) selalu dieksekusi terlebih dahulu untuk menginisialisasi bagian objek milik superclass. Pada eksperimen Step 6, pemanggilan `super(radius)` sengaja dihapus sementara dari constructor `Cylinder(radius, height)`. Akibatnya, Java secara implisit memanggil constructor tanpa parameter `Circle()`, sehingga nilai radius tetap bernilai default (1.0) dan bukan 2.0.



```

PS E:\Semester 3\PBO\Pertemuan-4\PBO-4> e.; cd 'e:\Semester 3\PBO\Pertemuan-4\PBO-4'; & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' 'cp' 'E:\Semester 3\PBO\Pertemuan-4\PBO-4\bin' 'TestCylinder'
Cylinder(double,double) dipanggil
c3
radius = 1.0
height = 10.0
base area = 3.141592653589793
volume = 31.41592653589793
PS E:\Semester 3\PBO\Pertemuan-4\PBO-4>

```

Gambar A.2: Bukti Constructor Chain (Radius tetap 1.0 saat `super(radius)` dihapus)

4. Bukti Batas Enkapsulasi (Akses Private Ditolak)

Inheritance tidak menembus batas aksesibilitas private. Pada eksperimen Step 7, dicoba akses langsung field private `radius` dari dalam method `cobaAksesRadius()` di class `Cylinder`. Compiler Java menolak kompilasi dengan pesan error: `The field Circle.radius is not visible`. Hal ini membuktikan bahwa subclass tidak dapat mengakses langsung field private superclass dan wajib menggunakan method accessor `getRadius()`.



Gambar A.3: Pesan Error Compiler Saat Subclass Mengakses Field Private Superclass

5. Override getArea(), Bug getVolume(), dan Perbaiki super.getArea()

Pada Step 8, makna getArea() pada Cylinder dioverride untuk menghitung total luas permukaan silinder:

- **Rumus Luas Permukaan:** $2 * \text{Math.PI} * \text{getRadius()} * \text{height} + 2 * \text{Math.PI} * \text{getRadius()} * \text{getRadius()};$

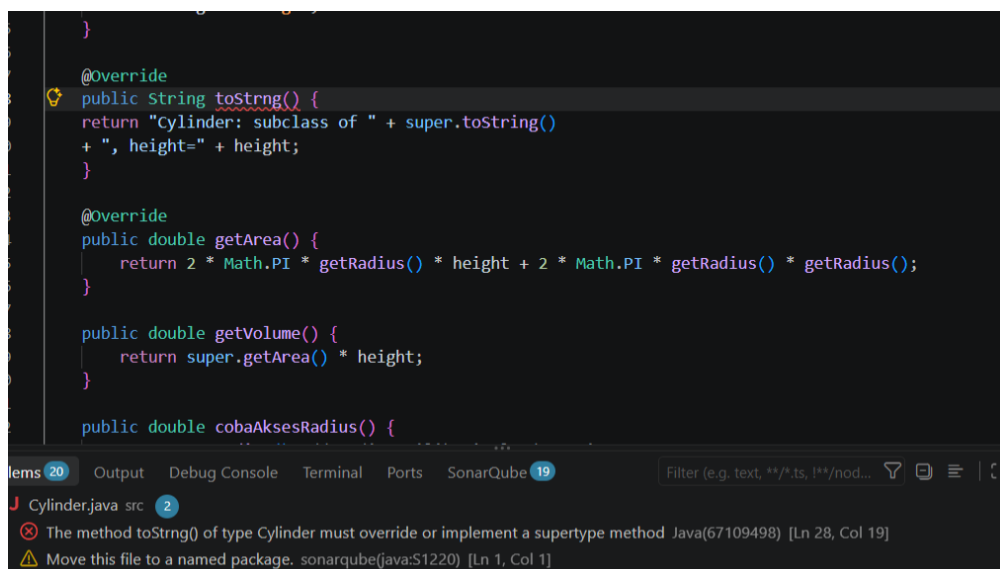
Hal ini menimbulkan konsekuensi pada method getVolume(). Rumus volume silinder adalah Luas Alas $\times$ Tinggi ($\pi r^2 \times h$). Jika getVolume() memanggil getArea() * height tanpa kualifikasi, method tersebut secara keliru mengalikan luas permukaan silinder dengan tinggi.

Solusi: getVolume() diperbaiki dengan secara eksplisit memanggil implementasi method milik superclass menggunakan super.getArea() * height, sehingga rumus luas lingkaran milik Circle yang dieksekusi.

6. Override toString() dan Uji Validasi Anotasi @Override

Method toString() pada Cylinder dioverride untuk menghasilkan informasi representasi teks silinder dengan menyertakan hasil pemanggilan super.toString().

Pada pengujian Step 9, nama method sengaja diubah menjadi toStrng() dengan tetap mempertahankan anotasi @Override. Compiler Java mendeteksi kesalahan tersebut dan menghentikan kompilasi karena method tersebut tidak meng-override method apa pun dari superclass. Ini membuktikan fungsi @Override sebagai mekanisme proteksi dini terhadap kesalahan ketik.



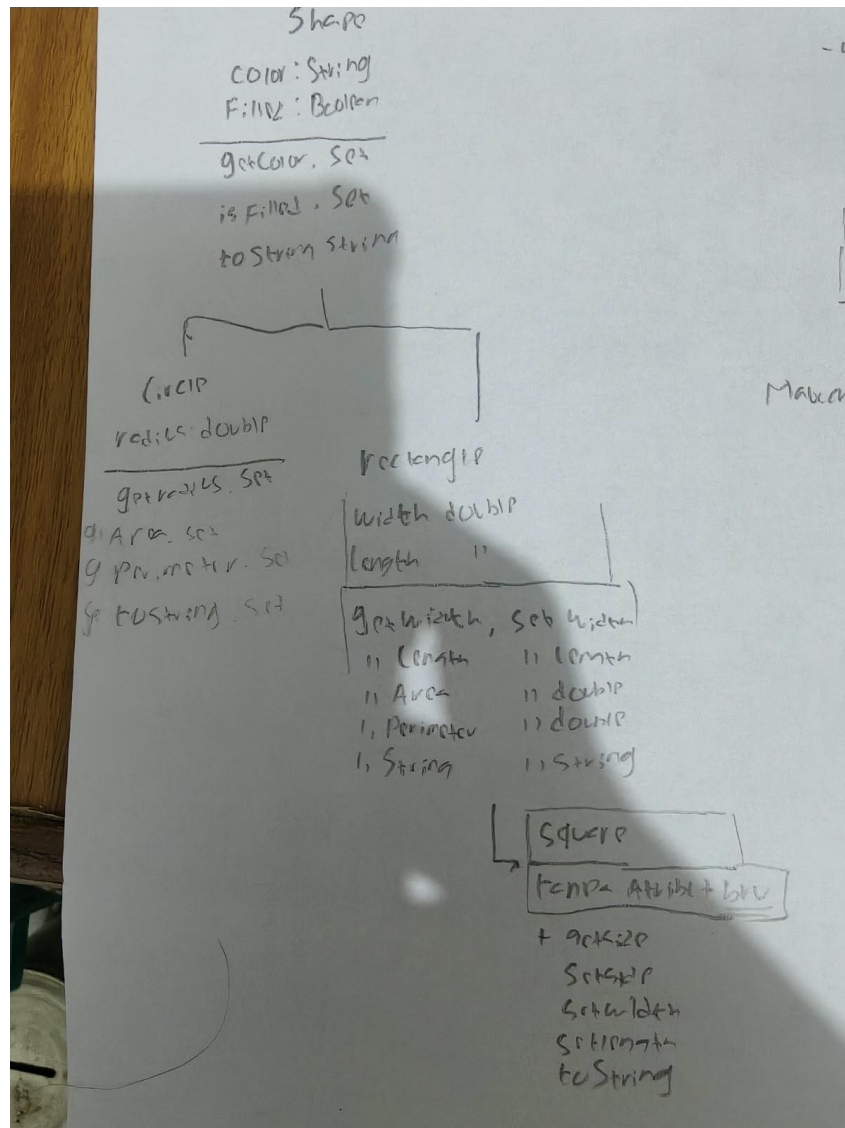
Gambar A.4: Error Compiler Akibat Salah Penulisan Nama Method Beranotasi @Override

B. Bagian B - Shape Hierarchy

Pada bagian ini, dibangun hierarki kelas geometri polimorfik bertingkat: Shape sebagai superclass umum, Circle dan Rectangle sebagai spesialisasi langsung dari Shape, serta Square sebagai spesialisasi dari Rectangle.

1. Diagram Hierarki Class (Shape, Circle, Rectangle, Square)

Diagram rancangan struktur hierarki kelas:



Gambar B.1: Diagram Hierarki Class Bangun Datar

2. Potongan Kode Inti Hierarki Shape

Kode: *Shape.java*

```
public class Shape {
    private String color;
    private boolean filled;

    public Shape() {
```

```

        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    @Override
    public String toString() {
        return "Shape[color=" + color + ", filled=" + filled + "]";
    }
}

```

Kode: Rectangle.java

```

public class Rectangle extends Shape {
    private double width;
    private double length;

    public Rectangle() {
        super();
        width = 1.0;
        length = 1.0;
    }

    public Rectangle(double width, double length) {
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean filled) {
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getLength() { return length; }
    public void setLength(double length) { this.length = length; }
    public double getArea() { return width * length; }
    public double getPerimeter() { return 2 * (width + length); }

    @Override
    public String toString() {
        return "Rectangle[width=" + width + ", length=" + length + ", " +
        super.toString() + "]";
    }
}

```

Kode: Square.java

```
public class Square extends Rectangle {
    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() { return getWidth(); }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setWidth(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setLength(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public String toString() {
        return "Square[side=" + getSide() + ", " + super.toString() + "];"
    }
}
```

Kode: TestShape.java

```
public class TestShape {
    public static void main(String[] args) {
        Shape a = new Shape("black", false);
        Circle b = new Circle(2.0, "blue", true);
        Rectangle c = new Rectangle(3.0, 4.0, "yellow", true);
        Square d = new Square(5.0, "green", true);

        System.out.println(a);
        System.out.println(b + " area=" + b.getArea());
        System.out.println(c + " area=" + c.getArea());
        System.out.println(d + " area=" + d.getArea());
    }
}
```

3. Output Eksekusi TestShape

Hasil pengujian seluruh hierarki kelas Shape dieksekusi melalui program TestShape:

```
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0
PS E:\Semester 3\PBO\Pertemuan-4\4>
```

Gambar B.2: Output Eksekusi Program TestShape

4. Bukti Square Sebelum dan Sesudah Override Setter

Pada Step 14, sebelum method `setWidth()` dan `setLength()` dioverride pada class `Square`, pemanggilan `s.setWidth(8.0)` hanya mengubah lebar menjadi 8.0 sementara panjang tetap 5.0. Akibatnya objek melanggar batasan geometri persegi (berubah menjadi persegi panjang).

Pada Step 15, kedua setter dioverride sehingga pemanggilan `setWidth(side)` maupun `setLength(side)` secara bersamaan memperbarui kedua atribut superclass via `super.setWidth(side)` dan `super.setLength(side)`. Dengan demikian, sifat simetri persegi selalu terjaga.

C. Bagian C - Pengembangan Kasus Tugas Sebelumnya (Kantin Polban)

Pada bagian ini, konsep inheritance diterapkan pada kasus tugas pertemuan sebelumnya, yaitu sistem pemesanan Kantin Polban.

1. Alasan Memilih Inheritance dan Analisis Generalisasi

Pada implementasi awal (Pertemuan 3), seluruh menu kantin direpresentasikan oleh satu class `MenuItem` tunggal tanpa membedakan kategori makanan dan minuman.

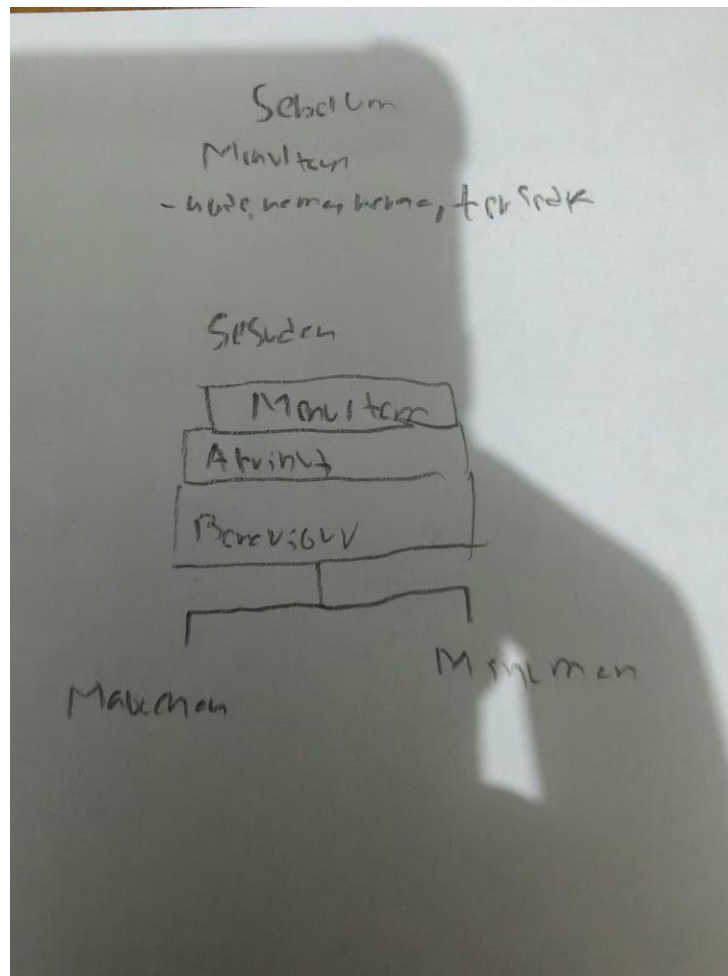
Berdasarkan analisis generalisasi (Step 17):

- **Atribut dan Behavior Bersama:** kode, nama, harga, tersedia, `tandaiHabis()`, dan perhitungan total.
- **Atribut dan Behavior Spesifik:** Makanan memiliki `levelPedas (int)`, sedangkan Minuman memiliki opsi suhu penyajian dingin (`boolean`).
- **Kaidah Relasi IS-A:** "Makanan adalah sebuah `MenuItem`" (Valid) dan "Minuman adalah sebuah `MenuItem`" (Valid).

Penerapan inheritance memisahkan entitas ke dalam hierarki yang tepat sehingga kode superclass dapat digunakan kembali (`reusable`) dan memudahkan penambahan jenis menu baru tanpa mengubah struktur kode yang sudah ada.

2. Diagram Hierarki Class Sebelum dan Sesudah

Sketsa rancangan perubahan model sebelum dan sesudah refactoring inheritance:



Gambar C.1: Diagram Hierarki Menu Kantin Sebelum dan Sesudah

3. Implementasi Kode Inti (extends, super, @Override, super.method)

Kode: MenuItem.java (Superclass)

```

public class MenuItem {
    private String kode;
    private String nama;
    private int harga;
    private boolean tersedia;

    public MenuItem(String kode, String nama, int harga) {
        this.kode = kode;
        this.nama = nama;
        this.harga = harga;
        this.tersedia = true;
    }

    public String getKode() { return kode; }
    public String getNama() { return nama; }
    public int getHarga() { return harga; }
    public boolean isTersedia() { return tersedia; }
    public void tandaiHabis() { this.tersedia = false; }

    public String getDeskripsi() {
        return "[" + kode + "]" + nama + " - Rp" + harga;
    }
}

```

```

@Override
public String toString() {
    return getDeskripsi() + " (" + (tersedia ? "Tersedia" : "Habis") + ")";
}
}

```

Kode: Makanan.java (Subclass)

```

public class Makanan extends MenuItem {
    private int levelPedas;

    public Makanan(String kode, String nama, int harga, int levelPedas) {
        super(kode, nama, harga);
        this.levelPedas = levelPedas;
    }

    public int getLevelPedas() { return levelPedas; }
    public void setLevelPedas(int levelPedas) { this.levelPedas = levelPedas; }

    @Override
    public String getDeskripsi() {
        return super.getDeskripsi() + " (Level Pedas: " + levelPedas + ")";
    }
}

```

Kode: Minuman.java (Subclass)

```

public class Minuman extends MenuItem {
    private boolean dingin;

    public Minuman(String kode, String nama, int harga, boolean dingin) {
        super(kode, nama, harga);
        this.dingin = dingin;
    }

    public boolean isDingin() { return dingin; }
    public void setDingin(boolean dingin) { this.dingin = dingin; }

    @Override
    public String getDeskripsi() {
        return super.getDeskripsi() + " (" + (dingin ? "Dingin/Es" : "Hangat") +
        ")";
    }
}

```

4. Output Pengujian Sistem Kasus Sebelumnya (TestKantin)

Pengujian seluruh skenario kasus kantin diimplementasikan pada class TestKantin.java untuk memverifikasi fungsionalitas turunan sekaligus memastikan tidak ada regresi pada sistem pesanan dan kasir.

```

Kantin
PENGUJIAN KASUS SEBELUMNYA (KANTIN POLBAN) - INHERITANCE
Objek Makanan & Minuman berhasil dibuat via constructor.
Method warisan: Harga Ayam Geprek = Rp15000
Status tersedia awal: true
Status setelah tandaiHabis(): false
Method override getDeskripsi Makanan: [MK01] Ayam Geprek - Rp15000 (Level Pedas: 3)
Method override getDeskripsi Minuman: [MN01] Es Teh Manis - Rp5000 (Dingin/Es)
Hasil getDeskripsi memanfaatkan super.getDeskripsi():
[MK01] Ayam Geprek - Rp15000 (Level Pedas: 3)
Update level pedas via setter: Level 5
Perbedaan behavior dua subclass:
Makanan: [MK01] Ayam Geprek - Rp15000 (Level Pedas: 5) (Tersedia)
Minuman: [MN01] Es Teh Manis - Rp5000 (Dingin/Es) (Tersedia)

REGRESSION TEST SISTEM KANTIN LAMA
Pesanan #1 diproses: Pesanan #1 | Pemesan: Salman Alfarisi (251511030) | Menu: [MK01] Ayam Geprek - Rp15000 (Level Pedas: 5) x2 | Total: Rp30000
Pesanan #2 diproses: Pesanan #2 | Pemesan: Salman Alfarisi (251511030) | Menu: [MN01] Es Teh Manis - Rp5000 (Dingin/Es) x1 | Total: Rp5000
Pesanan #3 ditolak!
Total pesanan dibuat: 3
PS E:\Semester 3\PBO\Pertemuan-4\4\src\kasus_sebelumnya>

```

Gambar C.2: Output Eksekusi Pengujian Kasus Kantin

D. Tabel Skenario Uji T4-01 s.d. T4-07

Berikut adalah tabel evaluasi pengujian skenario wajib sesuai spesifikasi modul Bagian 5.4:

ID	Skenario Uji	Expected Result	Status	Bukti Pengujian
T4-01	Object subclass dibuat melalui constructor	Bagian superclass dan subclass terinisialisasi benar	PASS	Objek Makanan ("MK01", "Ayam Geprek", 15000, 3) dan Minuman ("MN01", "Es Teh Manis", 5000, true) berhasil dibuat.
T4-02	Method warisan dipanggil dari subclass	Behavior superclass tersedia tanpa copy code	PASS	Method getHarga(), isTersedia(), dan tandaiHabis() milik MenuItem berhasil dipanggil pada objek Makanan/Minuman.
T4-03	Method override dipanggil	Behavior subclass yang dijalankan	PASS	Pemanggilan getDeskripsi() menghasilkan teks spesifik level pedas pada Makanan dan status dingin pada Minuman.
T4-04	Override memakai super.method(...)	Bagian behavior superclass tetap digunakan	PASS	getDeskripsi() pada Makanan dan Minuman memanggil super.getDeskripsi() untuk format dasar [kode] nama - harga.
T4-05	Akses state mengikuti encapsulation	Tidak mengakses field private secara langsung	PASS	Perubahan levelPedas dilakukan melalui setter setLevelPedas(5) tanpa mengakses field private secara langsung.
T4-06	Dua subclass diuji	Perbedaan behavior terlihat pada output	PASS	Output terminal menampilkan perbedaan deskripsi: Makanan memuat (Level Pedas: 5) dan Minuman memuat (Dingin/Es).
T4-07	Regression tugas lama	Fungsi utama sebelumnya tetap berjalan	PASS	Alur pemesanan kantin (Pesanan dan Kasir) tetap berfungsi normal: Pesanan #1 dan #2 diproses, Pesanan #3 ditolak.

E. Jawaban Refleksi Praktikum (Bagian 7.3)

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?

Circle merupakan bangun geometri dua dimensi yang lebih mendasar dan hanya membutuhkan parameter radius untuk mendefinisikan bentuk serta luasnya. Sedangkan Cylinder adalah bangun ruang tiga dimensi yang memperluas konsep lingkaran dengan menambahkan dimensi tinggi. Karena konsep lingkaran dapat berdiri sendiri tanpa memerlukan atribut silinder, Circle layak menjadi superclass yang lebih umum.

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?

Secara praktis, extends memungkinkan objek Cylinder untuk langsung memanggil dan menggunakan method publik yang didefinisikan pada Circle (seperti `getRadius()` dan `getArea()`) tanpa harus menduplikasi atau menulis ulang kode implementasinya pada class Cylinder.

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?

Karena objek subclass secara struktural dibentuk di atas fondasi superclass. State yang didefinisikan oleh Circle harus dialokasikan dan diinisialisasi terlebih dahulu di memori agar entitas superclass valid, sebelum constructor Cylinder dapat menginisialisasi atribut spesifik tambahannya (height).

4. Mengapa `getVolume()` menjadi salah setelah `getArea()` dioverride?

Karena di dalam `getVolume()`, pemanggilan `getArea()` merujuk pada method yang telah dioverride pada class Cylinder (yaitu luas permukaan total tabung). Akibatnya, perhitungan volume menjadi keliru karena rumus volume silinder mensyaratkan Luas Alas Lingkaran dikalikan Tinggi, bukan Luas Permukaan dikalikan Tinggi.

5. Apa yang secara eksplisit Anda minta ketika menulis `super.getArea()` atau `super.toString()`?

Penulisan `super.` secara eksplisit menginstruksikan runtime Java untuk mengeksekusi implementasi method asli yang berada pada superclass langsung, mengabaikan implementasi method yang telah dioverride pada subclass saat ini.

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?

Karena bentuk persegi memiliki aturan geometris invariant di mana panjang dan lebar harus selalu bernilai sama. Setter bawaan dari Rectangle mengizinkan perubahan salah satu dimensi secara independen, yang jika tidak dioverride akan merusak konsistensi bentuk persegi menjadi persegi panjang.

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.

Tidak setiap kemiripan memerlukan inheritance. Penggunaan inheritance hanya tepat jika terdapat hubungan hierarki tipe data yang valid (relasi IS-A). Pada kasus kantin, class Makanan dan Minuman tepat meng-extends MenuItem karena keduanya memang merupakan item menu yang dijual. Sebaliknya, class BahanBaku meskipun memiliki atribut mirip seperti nama dan harga, tidak tepat meng-extends MenuItem karena bahan baku bukan merupakan item menu yang dapat dipesan oleh pelanggan.

8. Lengkapi kalimat: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."

"...tetapi untuk memodelkan hubungan hierarki tipe data (IS-A) secara tepat, menjaga aturan konsistensi pada tingkatan abstraksi yang berbeda, serta memungkinkan penggunaan kembali kode yang aman dan terstruktur."

F. Link Repositori GitHub

Seluruh source code program dan laporan tugas praktikum ini diunggah pada repositori GitHub:

<https://github.com/salfffariss/PBO-4-Inheritance>